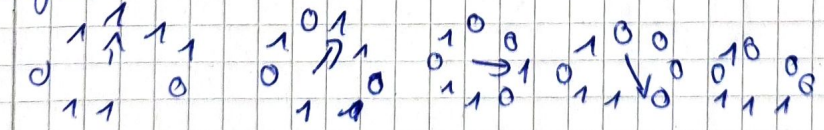
## Überschreiben von Einträgen in TLB/Caches:

1. Random verfahren
2. First in, First out (FIFO):
   Älteste Block wird ersetzt
3. Clock:
   Alle TLB/Cache-Einträge sind gedanklich in Kreis auf ein Zeitfensblatt angeordnet, ein Zeiger p wird in Uhrzeigersinn weiterbewegt und zeigt den zu ersetzenden Eintrag an.
   Jeder TLB/Cache-Eintrag hat ein used bit, das automatisch durch die TLB/Cache-Hardware auf '1' gesetzt wird, wenn eine auf den Eintrag zugegriffen wird.
   Das Clock-Verfahren sucht nach Eintrag mit used bit = '0' bei gleichzeitigem Rücksetzen des Bits.

Algorithmus: → Zeigt Zeiger p auf Eintrag mit used bit = '1'?
→ Falls ja: Setze used bit des Eintrages p auf '0'
bewege Zeiger um eine Position im Uhrzeigersinn weiter
(p+1)
→ Nein: Bewege Zeiger p+x und ersetze p-1

Bsp:

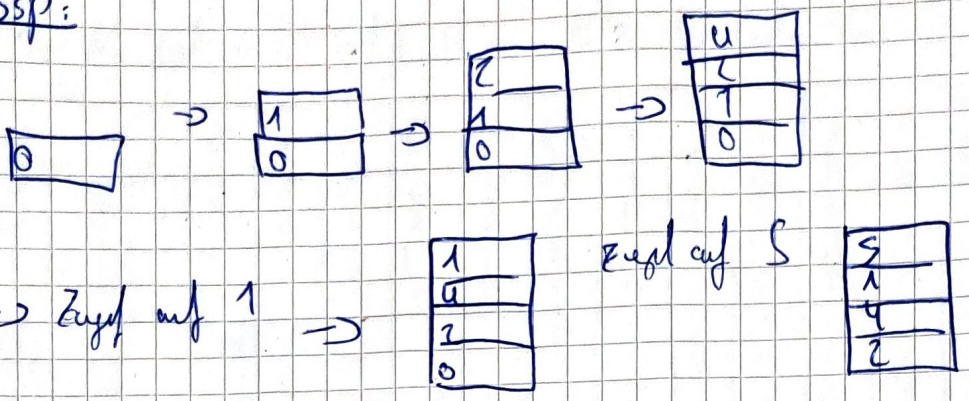


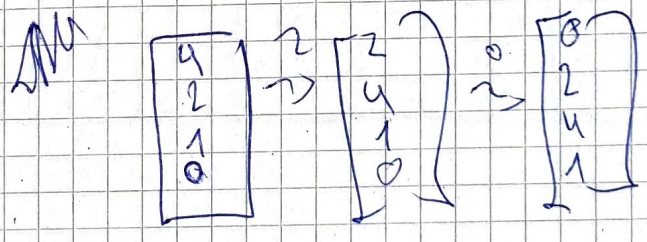
Technische Informatik Blatt 12

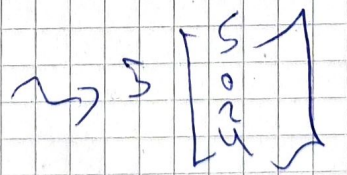
## 5. LRU, least recently used

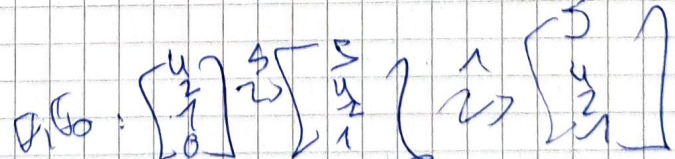
- Ersetze immer den Eintrag mit dem größten Rückwärtsabstand.

Bsp:



# Technische Informatik Blatt 12

(12-2)

Cache mit 4 Cache Lines

a) First in, First out (FIFO)

Der Hashwert entspricht dem Alter des Cache-Lines

| Referenzfolge | | E | A | F | D | A | B | A | B | C | A | F | B | E | D |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Cache | $B_0$ | E | E | E | E | E | B | B | B | B | B | B | B | E | F |
| | $B_1$ | | A | A | A | A | A | A | A | C | C | C | C | C | D |
| | $B_2$ | | | F | F | F | F | F | F | F | A | A | A | A | A |
| | $B_3$ | | | | D | D | D | D | D | D | D | F | F | F | E |
| Hash-wert | $B_0$ | 0 | 1 | 2 | 3 | 3 | 0 | 0 | 0 | 1 | 2 | 3 | 3 | 0 | 1 |
| | $B_1$ | | 0 | 1 | 2 | 2 | 3 | 3 | 3 | 0 | 1 | 2 | 2 | 3 | 0 |
| | $B_2$ | | | 0 | 1 | 1 | 2 | 2 | 2 | 3 | 0 | 1 | 1 | 2 | 3 |
| | $B_3$ | | | | 0 | 0 | 1 | 1 | 1 | 2 | 3 | 0 | 0 | 1 | 2 |

b) LRU (least Recently Used)

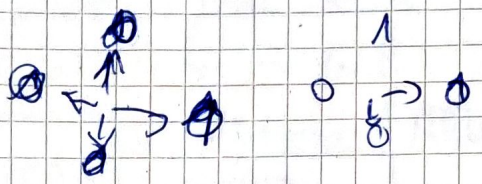
Der Hashwert entspricht dem Zugriffszeitpunkt.

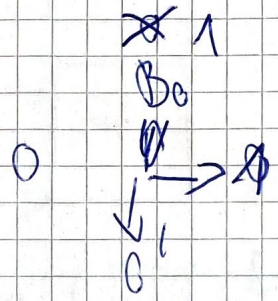
| Referenzfolge | | E | A | F | D | A | B | A | B | C | A | F | B | E | D |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Cache | $B_0$ | E | E | E | E | E | B | B | B | B | B | B | B | B | B |
| | $B_1$ | | A | A | A | A | A | A | A | C | C | C | C | E | D |
| | $B_2$ | | | F | F | F | F | F | F | F | A | A | A | A | A |
| | $B_3$ | | | | D | D | D | D | D | D | D | F | F | F | E |
| Hash-wert | $B_0$ | 0 | 1 | 2 | 3 | 3 | 0 | 1 | 0 | 1 | 2 | 3 | 0 | 1 | 2 |
| | $B_1$ | | 0 | 1 | 2 | 0 | 1 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 0 |
| | $B_2$ | | | 0 | 1 | 2 | 3 | 3 | 3 | 0 | 1 | 2 | 3 | 0 | 1 |
| | $B_3$ | | | | 0 | 1 | 2 | 2 | 2 | 3 | 3 | 0 | 1 | 2 | 3 |

# Technische Informatik B 6/11/12

c) Clock

| Reihenfolge | | E | A | F | D | A | B | A | B | C | A | F | B | E | D | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Code | $B_0$ | E | E ★ | E ★ | E ★ | E ★ | B ★ | B ★ | B ★ | B ★ | B ★ | B ★ | B ★ | E ★ | E ★ | |
| | $B_1$ | | | A | A | F | A | A | A | E | C | C | C | C | C | |
| | $B_2$ | | | F | F | D | F | F | F | ★ | ★ | F | F | F | F | |
| | $B_3$ | | | | D | D | D | D | D | D | D | F | F | F | F | |
| Wert | $B_0$ | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | |
| | $B_1$ | | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | |
| | $B_2$ | | | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | |
| | $B_3$ | | | | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | |
| Kürze | | $B_1$ | $B_2$ | $B_3$ | $B_0$ | $B_0$ | $B_1$ | $B_1$ | $B_1$ | $B_3$ | $B_3$ | $B_0$ | $B_0$ | $B_1$ | $B_2$ | |





Technische Informatik

## Cache

### Schreibverfahren

1. Write Through: Jeder Schreibvorgang in den Cache führt zu einer unmittelbaren Aktualisierung des Speichers

2. Copy-Back: Rückschreiben erfolgt erst, wenn Cache-Zeile bei Miss verdrängt wird

### Virtuelle & reale Caches

Virtuelle Caches (schnell):




Reale Caches (langsam):




Reale Caches (context switch): // Prozess P1 & P1 haben selbe virtuelle Adr. → Problem!

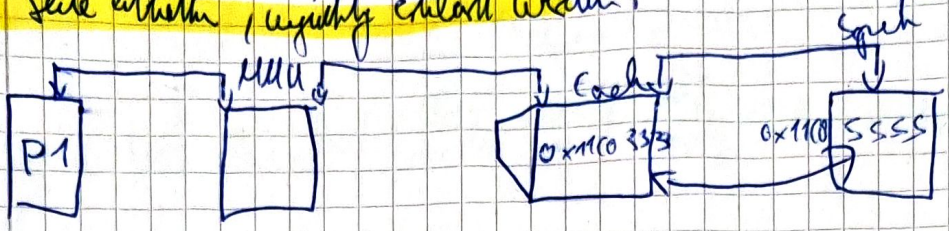
Reale Caches (bei Seitenfehler): // Adr. ändert sich

Bei Seitenfehlern müssen alle Cache-Zeilen die Kopien der verdrängten Seite enthalten, ungültig erklärt werden

# Technische Informatik – Cache

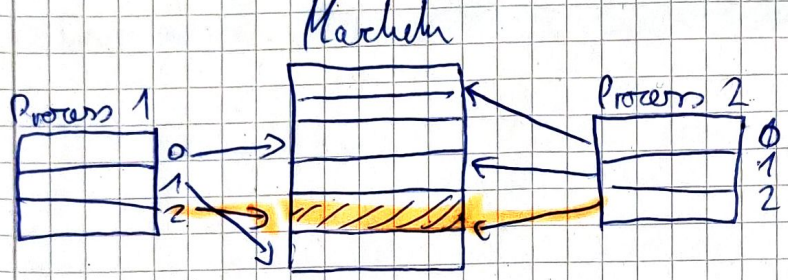
## Bus Snooping:

Bei Schreiben müssen alle Caches, die Kopien der veränderten Zeile enthalten, ungültig erklärt werden.




Cache kennzeichnet ob ein Schreibvorgang in den Speicher eine Zeile betrifft, von der er eine Kopie hat.

## Gemeinsam genutzte Seiten (Sharing)




Problem Cache bei Sharing: Nach Prozesswechsel erhält P2 aus dem Cache die nicht gültigen Informationen

# Technische Informatik

## Cache

### Virtueller Cache (context switch)

Zu einer bestimmten Adresse im Cache gehört von mal zu mal verschiedene Prozesswechsel eine andere Zelle im Speicher

-> Bei Prozesswechsel muss der Cache ungültig erklärt werden!

### Virtueller Cache bei Seitenfehler

-> keine Aktion erforderlich!

### Virtueller Cache bei Sperre

-> Der zweite Prozess füllt den Cache erneut, langsam aber sicher!

# Technische Informatik Blatt 13

## (13-1) SATA & USB

1 1 0 1 0 0 0 1 0 1 1 1 0 1 1 1 1 1 1 0 0 0 1 1 0 1 1 1 0 1

a) Welche Codierung wird bei SATA eingesetzt? Geben Sie den Signalverlauf an, der durch das vollständig korrekte SATA Signal erzeugt wird.

1 1 0 1 0 0 0 1 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 0 1 1 1 0 1



NRZ-Return-To-Zero (NRZ): Jedem Bitwert ist ein fester Signalpegel zugeordnet

b) Welche Codierung wird bei USB eingesetzt? Welches Problem löst es? Geben Sie den Signalverlauf an, der durch das vollständig korrekte USB-Signal erzeugt wird.

1 1 0 1 0 0 0 1 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 1 0 1 1 1 0 1



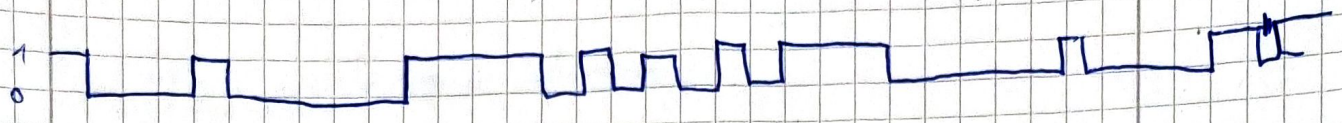
NRZI-Return-To-Zero-Inverted (NRZI): Signalpegel wechselt bei der Übertragung einer 0 und bleibt bei der Übertragung einer 1. Es löst das Synchronisationsproblem von NRZ, wenn zu viele Nullen bzw. Einsen am Stück gesendet werden.

# Technische Informatik – Übungsaufgabe

(10)

a)

1 0 1 1 0 0 1 1 1 0 1 1 1 0 0 0 0 0 0 0 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 0 1



b) Was bezeichnet NRZI, welches Problem löst es?

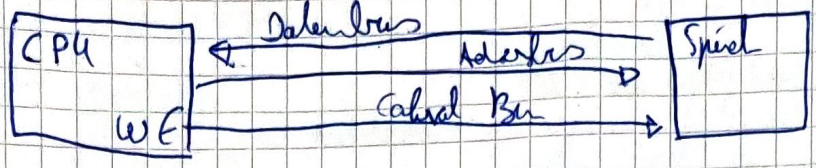
NON-Return-Zero-Inverted:
Bei jeder 0 wird der ~~Pegel~~ Signalwert invertiert und bei jeder 1 behält es bei. Aufeinander wird nach 6x 1 eine 0 hinzugefügt.

Es löst das Synchronisationsproblem, dass auftritt wenn bei NRZ zu viele 1 oder 0 hintereinander gesendet wird.

Z.B. 1 → Stuffing
⇒ 6x 1 → 0

# Rechnerarchitektur

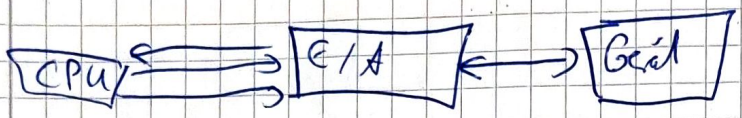
## I/O - aus Sicht der CPU




## Eigenschaften speicherbezogener Adressierung

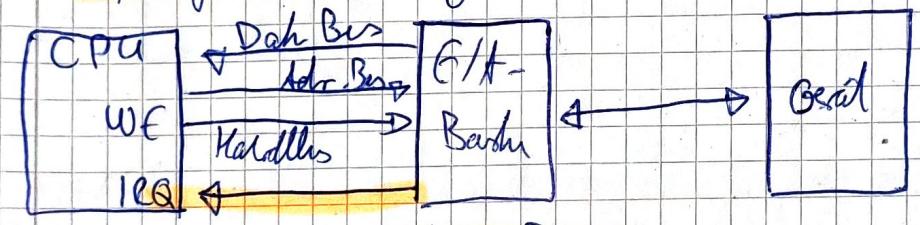
Vorteil: - Großer I/O-Adressraum aufgrundsam
- Wenig Festlegung für Prozessorentwurf
- Kein separater I/O-Befehle notwendig

Nachteil: - Besonderes I/O-Timing mit üb Busadapter
- Zugriffsschutz auf I/O Geräte nur über Speicherverwaltung




## Unterbrechungen (Interrupts)

- Verkehr von Gerät über den I/O-Baustein zur CPU
- Anforderung einer Behandlung (z.B Taste)




## Synchrone und asynchrone Busse

Klassifizierung basiert auf Unterscheidung zwischen unidirektionalem Timing (bei synchronen Bussen) und bidirektionalem Timing

### Unidirektionales Timing:

→ Kommunikationspartner verlassen sich darauf, dass Partner innerhalb festgelegter Zeit reagieren

### Bidirektionales Timing

- Kommunikationspartner bestätigen per Handshakesignal, dass sie in der erwarteten Weise reagiert haben

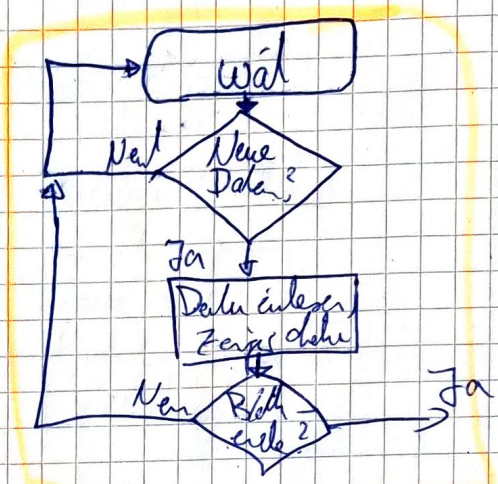
Technische Informatik

## Vergleich der Methoden

| | unidirektional | bidirektional |
|---|---|---|
| Vorteile | einfach; bei häufigem Datenwechsel schnell | passt sich unterschiedlichen Geschwindigkeiten an; hoher Grad an Flexibilität möglich |
| Nachteil | Kommunikationspartner muss in bestimmter Zeit antworten | komplexer; Zeit alternativ nötig, langsam |
| Einsatz-gebiet | synchrone Busse, Speicherbusse | asynchrone Busse I/O - Bus |

### Status-Methode (busy waiting)

Das Gerät baut die Steuerung stellt Bereit-Bit zur Verfügung




Nachteile: - keine Möglichkeit der verzahnten Bearbeitung anderer Aufgaben
- geringe Übertragungsgeschwindigkeit

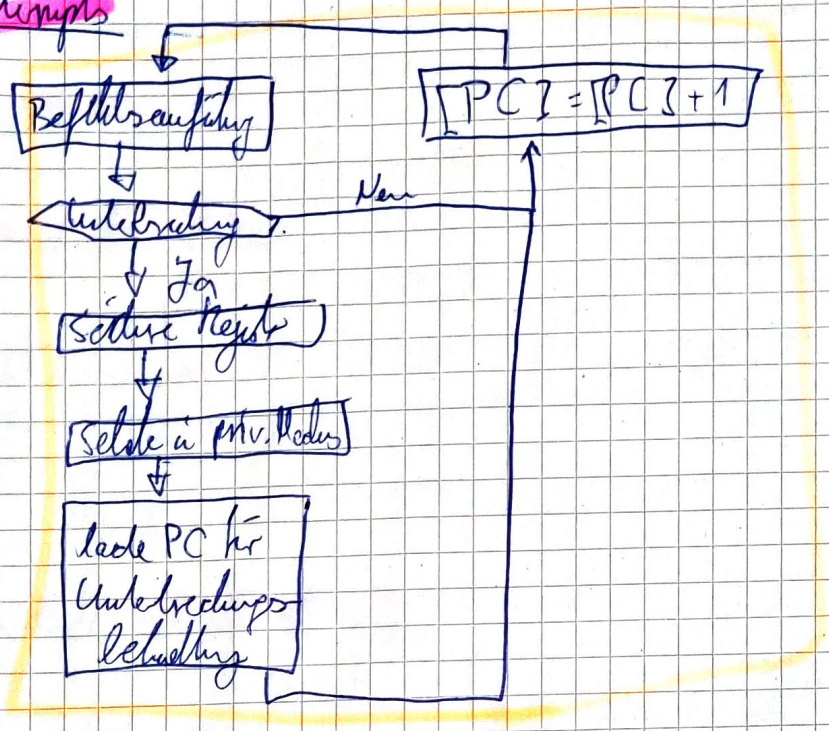
# Technische Informatik

## Polling

Polling = gelegentliches Prüfen des Bereit Bits mit verschiedener Bearbeitung oder Aufgaben




## Interrupts




Ablauf: → unterbrochener Befehl
↻ Interrupt

## Sinn & Zweck

- Gerätabfrage
- Behandlung externer Ereignisse

# Technische Informatik

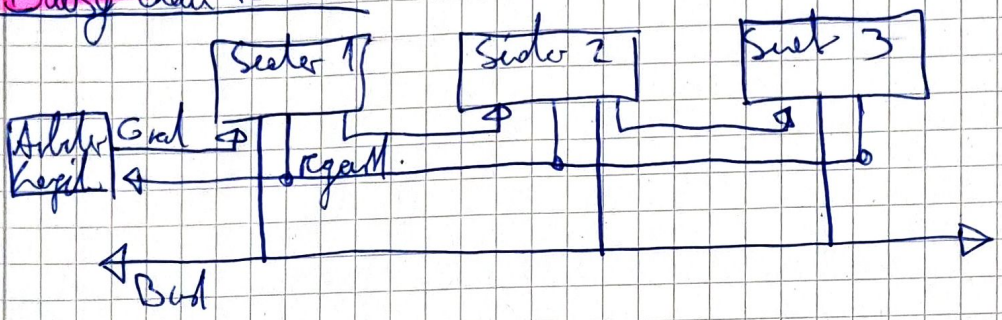
## Direct Memory Access, DMA

Bei DMA wird die Datentransportleistung zwischen Geräte-Steuerung und Speicher durchgeführt, ohne CPU.

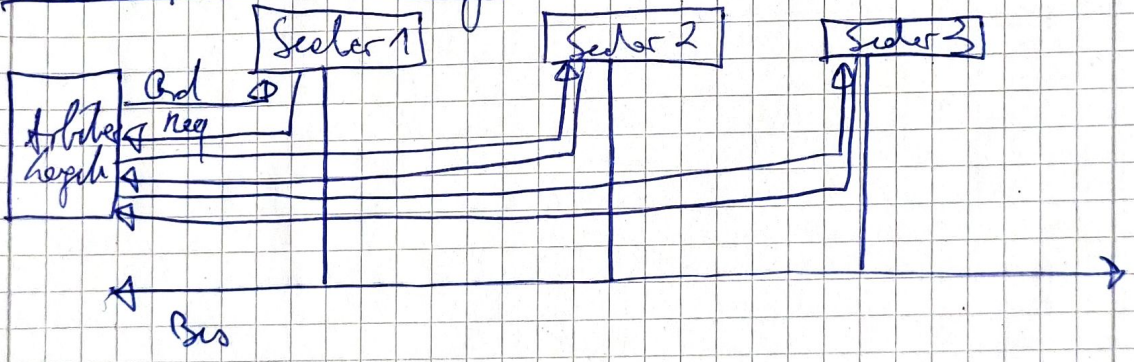
⇒ DMA macht es erforderlich, dass Geräte-Steuerungen Bus-Master werden können

## Bus-Zugriff

### Daisy chain Technik




### Parallele, zentrale Arbitrierung



# Technische Informatik

**Busarbitierung:**
ist ein Verfahren, welches entscheidet, welcher Bus-Teilnehmer den Bus exklusiv nutzen darf.
⚠ Folgende Probleme werden verhindert:
→ elektrische Kurzschlüsse
→ Datenkollisionen